
Práctica Guiada: Administrando Contenedores y Servicios Web con Docker y Docker Compose

Implantación de Aplicaciones Web

índice

FASE 1: Primeros Pasos con Apache en Docker

 Capturas requeridas para la memoria:

FASE 2: Creación de Imagen Personalizada y Docker Hub

 Capturas requeridas para la memoria:

FASE 3: Laboratorio de Diagnóstico y Resolución de Errores

Escenario A: Conflicto de Puertos

Escenario B: Ruta Incorrecta en el Dockerfile

Escenario C: Caída Inmediata del Contenedor (Crash)

 Capturas requeridas para la memoria:

FASE 4: Orquestación con Docker Compose

 Capturas requeridas para la memoria:

Práctica: Docker

- Operar contenedores desde la CLI aplicando mapeo de puertos y gestión de procesos.
- Diseñar e interpretar un **Dockerfile** para construir una imagen personalizada basada en **Apache (httpd)**.
- Publicar e interactuar con imágenes en el registro público **Docker Hub**.
- Diagnosticar y resolver fallos comunes mediante el análisis de logs y estado de contenedores.
- Automatizar el despliegue de múltiples servicios utilizando **Docker Compose**.

FASE 1: Primeros Pasos con Apache en Docker

Reto: Debes poner en marcha un servidor web Apache ligero utilizando la versión **alpine** de la imagen oficial.

1. Despliegue inicial:

- Busca en Docker Hub la imagen oficial de Apache (**httpd**) en su variante **alpine**.
- Lanza un contenedor en segundo plano que redirija el puerto **8080 de tu equipo local** al puerto interno estándar donde escucha Apache (**80**).
- Asigna un nombre identificativo a tu contenedor.

2. Comprobación y Registros (Logs):

- Accede desde tu navegador web a la dirección local correspondiente y confirma que el servidor responde.
- Visualiza los logs en tiempo real mediante la consola de Docker y genera peticiones en el navegador para verificar cómo se registran los accesos HTTP (códigos **200 OK**, etc.).



Capturas requeridas para la memoria:

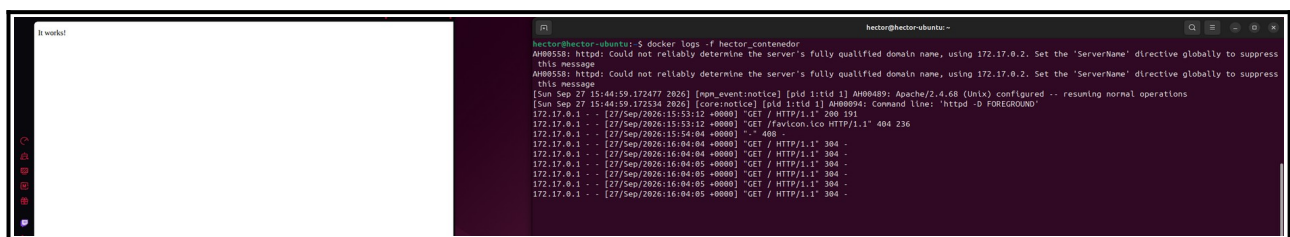
Captura 1.1: Salida de la consola con la ejecución del comando para arrancar el contenedor y listado del contenedor en ejecución (**ps**).

```

hector@hector-ubuntu: ~
hector@hector-ubuntu:~$ docker run -d -p 8080:80 --name hector_contenedor httpd:alpine
Unable to find image 'httpd:alpine' locally
alpine: Pulling from library/httpd
e2de96513ba9: Pull complete
e2e5f9b3fd79: Pull complete
6c7e7d1981af: Pull complete
36661b309997: Pull complete
ddb55e31b4d3: Pull complete
4f4fb708ef54: Pull complete
8210c35d0f11: Pull complete
9d1880b9f18d: Download complete
6e0336dae256: Download complete
Digest: sha256:4e585da9d0125dec36d4500a9f5c5df7b2c0a01f67cb47865a91a4b05bdbc1b
Status: Downloaded newer image for httpd:alpine
0325598bccd40a4574a44571d680564ee0547418a440c1566021d3bc2f492ba9
hector@hector-ubuntu:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0325598bccd4   httpd:alpine   "httpd-foreground"      22 seconds ago Up 21 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   hector_contenedor
hector@hector-ubuntu:~$

```

Captura 1.2: Navegador web cargando la página por defecto junto a la terminal mostrando los registros del servidor (**logs**).



FASE 2: Creación de Imagen Personalizada y Docker Hub

Reto: Crearás un sitio web corporativo propio dentro del contenedor, empaquetarás el resultado como una imagen reutilizable y la publicarás en tu cuenta personal de Docker Hub.

1. **Construcción de la Web y el Dockerfile:**
 - Crea un directorio de trabajo.

- Crea un archivo `index.html` con una estructura HTML básica que incluya tu nombre, fecha y un título.
- Escribe un archivo `Dockerfile` que utilice como base la imagen `httpd:alpine`, copie tu archivo `index.html` a la **ruta interna donde Apache busca las páginas por defecto**, y exponga el puerto web.

💡 **Pista para la ruta de Apache:** En las imágenes oficiales basadas en Alpine, el directorio raíz del servidor web no suele ser `/var/www/html/`, sino `/usr/local/apache2/htdocs/`.

2. Publicación en Docker Hub:

- Regístrate/inicia sesión en [Docker Hub](#).
- Inicia sesión desde tu terminal.
- Construye tu imagen asignándole una etiqueta (*tag*) que incluya tu nombre de usuario de Docker Hub (ejemplo: `tu_usuario/mi-apache-web:v1.0`).
- Subela a tu repositorio remoto en Docker Hub.

3. Prueba de Descarga:

- Elimina la imagen local y el contenedor que creaste para hacer la prueba.
- Descarga (`pull`) tu imagen desde Docker Hub y vuelve a levantarla en un puerto distinto (ejemplo: **8081**).



Capturas requeridas para la memoria:

Captura 2.1: Contenido del archivo `Dockerfile`.

```
hector@hector-ubuntu: ~/hector_docker
GNU nano 7.2 Dockerfile
FROM httpd:alpine
COPY index.html /usr/local/apache2/htdocs
EXPOSE 80
```

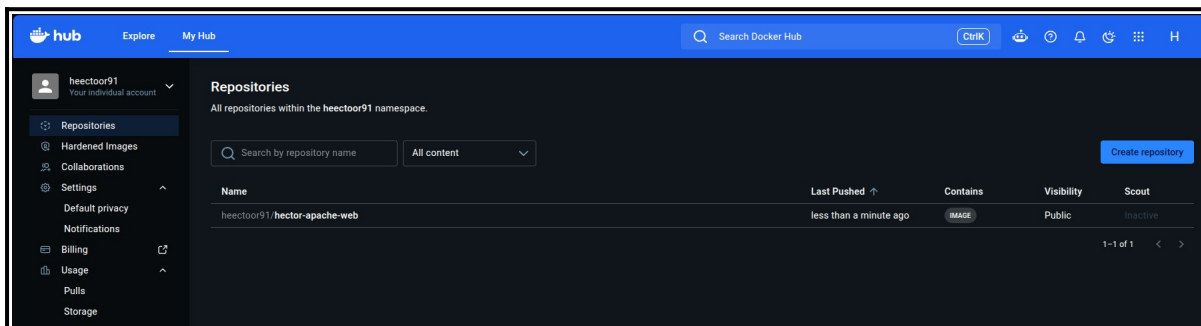
Captura 2.2: Proceso de subida (`push`) completado con éxito en la consola.

```

hector@hector-ubuntu:~/hector_docker$ docker push heectoor91/hector-apache-web:v1.0
The push refers to repository [docker.io/heectoor91/hector-apache-web]
44136fa355b3: Pushed
36661b309997: Mounted from library/httpd
e2de96513ba9: Mounted from library/httpd
7572d31a1979: Pushed
ddb55e31b4d3: Mounted from library/httpd
4f4fb700ef54: Mounted from library/httpd
8210c35d0f11: Mounted from library/httpd
6c7e7d1981af: Mounted from library/httpd
e2e5f9b3fd79: Mounted from library/httpd
04a77803624a: Pushed
v1.0: digest: sha256:9569b4ea52613614ab00a832afa6b6c481e6d9951d098c289b996075ff86684f size: 856
hector@hector-ubuntu:~/hector_docker$

```

Captura 2.3: Tu repositorio público visible en la web de Docker Hub.



Captura 2.4: Ejecución del contenedor descargado desde Docker Hub funcionando en el navegador.



FASE 3: Laboratorio de Diagnóstico y Resolución de Errores

Reto: En esta fase te enfrentarás a problemas reales. Tendrás que provocar los fallos, analizar la información disponible y explicar cómo los has solucionado.

Escenario A: Conflicto de Puertos

1. Intenta ejecutar un nuevo contenedor asignándole en el equipo anfitrión el mismo puerto que ya está utilizando otro contenedor activo.
2. Lee el error devuelto por la terminal.
3. Explicación requerida: ¿Por qué ocurre y qué comando o parámetro debes modificar para solventarlo? → Porque un puerto solo puede escuchar 1 aplicación y ya hay un contenedor ocupando ese puerto. Para solventarlo hay que cambiar el puerto, por ejemplo, 8082 o 8084

Escenario B: Ruta Incorrecta en el Dockerfile

1. Modifica tu **Dockerfile** introduciendo una ruta de destino intencionadamente errónea para el comando de copia del HTML (por ejemplo, `/var/www/html/`).
2. Construye y ejecuta la imagen.
3. Al acceder al navegador notarás que la web no muestra tu archivo personalizado.
4. Inspección requerida: Accede de forma interactiva a la consola interna del contenedor o inspecciona la ruta con comandos de Docker para verificar qué archivos hay en la ruta oficial de Apache.

Escenario C: Caída Inmediata del Contenedor (*Crash*)

1. Modifica la instrucción de arranque por defecto (**CMD** o **ENTRYPOINT**) de un **Dockerfile** escribiendo un parámetro o comando que no exista.
2. Intenta ejecutar el contenedor en segundo plano (**-d**). Notarás que el contenedor se detiene inmediatamente.
3. Diagnóstico requerido: Identifica el estado del contenedor usando el flag adecuado en el comando de listado e inspecciona los registros para determinar la causa exacta de la parada.



Capturas requeridas para la memoria:

Captura 3.1: Mensaje de error de puerto duplicado en la terminal.

```
hector@hector-ubuntu:~/hector_docker$ docker run -d -p 8081:80 --name contenedor-error-puerto httpd:alpine
e5dceadd15f28f945fae58ba98669e76b487f13ae2f8cf9ec4287dcc2226b357
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint contenedor-error-puerto (b19ab58554
aadef253f7f3d3fec7d0701c13774aae5306d3adcd48b5e3d5755): Bind for :::8081 failed: port is already allocated

Run 'docker run --help' for more information
hector@hector-ubuntu:~/hector_docker$
```

Captura 3.2: Salida de comandos o acceso interactivo que demuestre dónde se guardó la web incorrectamente en el Escenario B.

```
hector@hector-ubuntu:~/hector_docker$ docker exec -it contenedor-mal-ruta sh
/usr/local/apache2 # ls /var/www/html
index.html
/usr/local/apache2 #
```

Captura 3.3: Estado del contenedor detenido (**Exited**) y lectura del error mediante **docker logs** en el Escenario C.

```
hector@hector-ubuntu:~/hector_docker$ docker run -d --name contenedor-comando-error hector91/hector-apache-web:comando-error
265d0557ce41d5bfc9e8eb6c114080be3814db442b37c9d77b98129199e7aa
docker: Error response from daemon: failed to create task for container: failed to create shin task: OCI runtime create failed: runc create failed: unable to start container process: error during container init: exec: "comando_error": executable file not found in $PATH
Run 'docker run --help' for more information
hector@hector-ubuntu:~/hector_docker$
```

```
hector@hector-ubuntu:~/hector_docker$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
265d0557ce41	hector91/hector-apache-web:comando-error	"comando_error"	2 minutes ago	Created		contenedor-comando-erro
f330e19f8a884	hector91/hector-apache-web:version-erronea	"httpd-foreground"	15 minutes ago	Up 14 minutes	0.0.0.0:8084->80/tcp, [::]:8084->80/tcp	contenedor-mal-ruta
e5dceadd15f2	httpd:alpine	"httpd-foreground"	28 minutes ago	Created		contenedor-error-puerto
b2d6b2e36acd	hector91/hector-apache-web:v1.0	"httpd-foreground"	32 minutes ago	Up 32 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	hector_contenedor_2
0325598bccd4	httpd:alpine	"httpd-foreground"	3 hours ago	Up 3 hours	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	hector_contenedor
e6908482d77	hello-world	"/hello"	2 days ago	Exited (0) 2 days ago		strange_brown

```
hector@hector-ubuntu:~/hector_docker$ docker logs contenedor-comando-error
hector@hector-ubuntu:~/hector_docker$
```

FASE 4: Orquestación con Docker Compose

Reto: Diseñar una infraestructura como código donde pongas en marcha un entorno con dos servidores web Apache simultáneos en una sola instrucción.

1. Diseño del archivo **docker-compose.yml**:

- Crea un archivo **docker-compose.yml** en la raíz de tu proyecto.
- Define una versión de sintaxis adecuada.
- Declara dos servicios:
 - **Servicio 1 ("web-personalizada"):** Debe construirse a partir del **Dockerfile** que creaste en la Fase 2 y exponerse en el puerto **8000**.
 - **Servicio 2 ("web-oficial"):** Debe utilizar la imagen directa **httpd:alpine** de Docker Hub y exponerse en el puerto **8001**.
- Define políticas de reinicio (**restart**) para ambos servicios.

2. Despliegue y Control:

- Arranca ambos servicios en segundo plano utilizando únicamente un comando de Docker Compose.
- Lista los servicios activos pertenecientes a la orquestación.
- Muestra los registros combinados de ambos servicios en tiempo real.
- Detén y elimina toda la infraestructura orquestada con un solo comando.



Capturas requeridas para la memoria:

Captura 4.1: Contenido completo del archivo `docker-compose.yml`.

```

docker-compose.yml x
home > hector > hector_docker > docker-compose.yml
1  version: '3.8'
2
3  services:
4    web-personalizada:
5      build: .
6      ports:
7        - "8000:80"
8      restart: always
9
10   web-oficial:
11     image: httpd:alpine
12     ports:
13       - "8001:80"
14     restart: always
  
```

Captura 4.2: Salida del terminal al ejecutar la orden de arranque del Compose y la comprobación de estado de los servicios.

```

hector@hector-ubuntu:~/hector_docker$ docker compose up -d
WARN[0000] /home/hector/hector_docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 0.4s (9/9) FINISHED
=> [internal] load local bake definitions                                0.0s
=> reading from stdin 5000                                             0.0s
=> [internal] load build definition from Dockerfile                    0.0s
=> == transferring dockerfile: 107B                                     0.0s
=> [internal] load metadata for docker.io/library/httpd:alpine         0.0s
=> [internal] load .dockerignore                                         0.0s
=> == transferring context: 2B                                           0.0s
=> [internal] load build context                                         0.0s
=> == transferring context: 32B                                          0.0s
=> [1/2] FROM docker.io/library/httpd:alpine@sha256:4e585da9d0125dec36d4580a9f5c5df7b2c0a01f67cb47865a91a4b85bdbc1b
=> == resolve docker.io/library/httpd:alpine@sha256:4e585da9d0125dec36d4580a9f5c5df7b2c0a01f67cb47865a91a4b85bdbc1b
=> CACHED [2/2] COPY index.html /usr/local/apache2/htdocs              0.0s
=> exporting to image                                                  0.1s
=> == exporting layers                                                  0.0s
=> == exporting manifest sha256:4bd0717e537490a2e10649a584a972bb007d4d1e1b4b66eb52c42b69c01ad2 0.0s
=> == exporting config sha256:ef62272570cc1c0a150a3eb71e0f2fba3eef1776a44a660595d6197c445c1a3 0.0s
=> == exporting attestation manifest sha256:10e2df4f8bb09ce2e02b87f1e64d2ba5f5688bac87f4067ec7925e7727ab3a01 0.0s
=> == exporting manifest list sha256:6ef27a3ab9f8c112a1e2b1a8fdaf041e54d29f5e07be43e5ceea1d5e92e4 0.0s
=> == naming to docker.io/library/hector_docker-web-personalizada:latest 0.0s
=> == unpacking to docker.io/library/hector_docker-web-personalizada:latest 0.0s
=> == resolving provenance for metadata file                             0.0s
[+] up 4/4
✔ Image hector_docker-web-personalizada Built 0.5s
✔ Network hector_docker_default Created 0.0s
✔ Container hector_docker-web-personalizada-1 Started 0.5s
✔ Container hector_docker-web-oficial-1 Started 0.3s
  
```

Captura 4.3: Ambos sitios web (puerto 8000 y puerto 8001) cargados simultáneamente en pestañas del navegador.

